

1. CTRL.v

기본적으로 opcode에 따라서 각 control의 값을 정하는 코드이다. R-type이라면, 추가적으로 funct에 따라 ALUOp의 값을 변경해 주었다.

```
module CTRL(
    always @(*) begin
        case (opcode)
            `OP_RTYPE: begin
                case (funct)
                    end
                    default: begin
                        RegDst = 1;
                        RegWrite = 1;
                        case (funct)
                            `FUNCT_SLL: ALUOp = `ALU_SLL;
                            `FUNCT_SRL: ALUOp = `ALU_SRL;
                            `FUNCT_SRA: ALUOp = `ALU_SRA;
                            `FUNCT_ADDU: ALUOp = `ALU_ADDU;
                            `FUNCT_SUBU: ALUOp = `ALU_SUBU;
                            `FUNCT_AND: ALUOp = `ALU_AND;
                            `FUNCT_OR: ALUOp = `ALU_OR;
                            `FUNCT_XOR: ALUOp = `ALU_XOR;
                            `FUNCT_NOR: ALUOp = `ALU_NOR;
                            `FUNCT_SLT: ALUOp = `ALU_SLT;
                            `FUNCT_SLTU: ALUOp = `ALU_SLTU;
                            default: ALUOp = `ALU_ADDU;
                        endcase
                    end
                endcase
            end
            `OP_J: begin
                Jump = 1;
            end
            `OP_JAL: begin
                Jump = 1;
                RegWrite = 1;
                SavePC = 1;
            end
            `OP_BEQ: begin
                Branch = 1;
                ALUOp = `ALU_EQ;
            end
            `OP_BNE: begin
                Branch = 1;
                ALUOp = `ALU_NEQ;
            end
            `OP_ADDIU: begin
                ALUSrc = 1;
                SignExtend = 1;
                RegWrite = 1;
                ALUOp = `ALU_ADDU;
            end
            `OP_SLTI: begin
                ALUSrc = 1;
                SignExtend = 1;
                RegWrite = 1;
                ALUOp = `ALU_SLT;
            end
            `OP_SLTIU: begin
                ALUSrc = 1;
                SignExtend = 1;
                RegWrite = 1;
                ALUOp = `ALU_SLTU;
            end
            `OP_ANDI: begin
                ALUSrc = 1;
                RegWrite = 1;
                ALUOp = `ALU_AND;
            end
            `OP_ORI: begin
                ALUSrc = 1;
                RegWrite = 1;
                ALUOp = `ALU_OR;
            end
            `OP_XORI: begin
                ALUSrc = 1;
                RegWrite = 1;
                ALUOp = `ALU_XOR;
            end
            `OP_LUI: begin
                ALUSrc = 1;
                SignExtend = 1;
                RegWrite = 1;
                ALUOp = `ALU_LUI;
            end
            `OP_LW: begin
                ALUSrc = 1;
                SignExtend = 1;
                RegWrite = 1;
                MemRead = 1;
                MemtoReg = 1;
                ALUOp = `ALU_ADDU;
            end
            `OP_SW: begin
                ALUSrc = 1;
                SignExtend = 1;
                MemWrite = 1;
                ALUOp = `ALU_ADDU;
            end
        end
    end
end
```

2. CPU.v

기본적으로 3가지 부분으로 나누어 작성했다.

우선 wire들을 할당해주는 부분이다.

```
// Define the wires

assign halt          = (inst == 32'b0);

assign opcode        = inst[31:26];
assign rs            = inst[25:21];
assign rt            = inst[20:16];
assign rd            = inst[15:11];
assign shamt         = inst[10:6];
assign funct         = inst[5:0];
assign immi          = inst[15:0];
assign immj          = inst[25:0];

assign ext_imm       = SignExtend ? {{16{immi[15]}}, immi} : {16'b0, immi};

assign rd_addr1      = rs;
assign rd_addr2      = rt;

assign operand1      = rd_data1;
assign operand2      = ALUSrc ? ext_imm : rd_data2;

assign mem_addr      = alu_result;
assign mem_write_data = rd_data2;
```

두번째는 always @(*) begin 문이다.

```
always @(*) begin
    if (SavePC)                wr_addr = 5'd31;
    else if (RegDst)           wr_addr = rd;
    else                       wr_addr = rt;

    if (SavePC)                wr_data = PC + 4;
    else if (MemtoReg)         wr_data = mem_read_data;
    else                       wr_data = alu_result;

    if (halt)                  PC_next = PC;
    else if (JR)                PC_next = rd_data1;
    else if (Branch & alu_result) PC_next = PC + 4 + {ext_imm[29:0], 2'b00};
    else if (Jump)              PC_next = {PC[31:28], immj, 2'b00};
    else                        PC_next = PC + 4;
end
```

마지막은 모듈의 정의 부분이다.

```
CTRL ctrl (  
    .opcode(opcode),  
    .funct(funct),  
    .RegDst(RegDst),  
    .Jump(Jump),  
    .Branch(Branch),  
    .JR(JR),  
    .MemRead(MemRead),  
    .MemtoReg(MemtoReg),  
    .MemWrite(MemWrite),  
    .ALUSrc(ALUSrc),  
    .SignExtend(SignExtend),  
    .RegWrite(RegWrite),  
    .ALUOp(ALUOp),  
    .SavePC(SavePC)  
);  
  
RF rf (  
    .clk(clk),  
    .rst(rst),  
    .rd_addr1(rs),  
    .rd_addr2(rt),  
    .rd_data1(rd_data1),  
    .rd_data2(rd_data2),  
    .RegWrite(RegWrite),  
    .wr_addr(wr_addr),  
    .wr_data(wr_data)  
);  
  
MEM mem (  
    .clk(clk),  
    .rst(rst),  
    .inst_addr(PC),  
    .inst(inst),  
    .mem_addr(mem_addr),  
    .MemWrite(MemWrite),  
    .mem_write_data(mem_write_data),  
    .mem_read_data(mem_read_data)  
);  
  
ALU alu (  
    .operand1(operand1),  
    .operand2(operand2),  
    .shamt(shamt),  
    .funct(ALUOp),  
    .alu_result(alu_result)  
);
```

3. 주의한 부분

우선 JAL과 JR 명령어의 구현 부분이다.

JAL의 경우 J와 동일하고 PC를 점프시키되, 추가로 PC + 4를 SavePC를 이용하여 ra레지스터에 저장한다. 이는 CPU.v의 두번째 파트 중 if(SavePC)에 나와있다. wr_addr에는 ra주소인 5'd31을, wr_data에는 PC + 4를 저장해 주었다.

JR은 R-type 명령어지만 ALU연산 결과가 아닌 rs 레지스터 값으로 PC를 저장한다. 이를 위하여 CTRL.v 내의 기존 case문에서 funct가 FUNCT_JR인 경우만 따로 처리를 해주었다.

다음으로는 SignExtend 부분이다

I-type 명령어인 ANDI, ORI, XORI의 경우 sign extension이 아닌 zero extension을 해주어야 했는데, 처음에는 어렵다고 생각했지만 immi[15]를 이용하여 생각보다 쉽게 구현이 가능했다.

4. 테스트 결과

테스트 결과, 7개의 테스트케이스 모두 문제 없이 통과했다.